Data analysis and biostatistics

Jordi Vivas Sakuta and Izan Flores Tortosa

Exercise 1

Write a code that stores in a vector all numbers from 1 to 100 but skips the numbers 3,9,13,19,23,29,33,39.... Tips: you can use the operator %in% to check if a value is in a vector. Use the function diff (difference/derivative) to find the pattern behind the forbidden numbers sequence. Check whether the result is correct with the function any. Tip: First, construct the forbidden numbers sequence.

```
#Write your code here
ve4<-NULL
ve<-c(1:100)
ve2<-ve[seq(from=3,to=100,by=10)]
ve3<-seq(from=9,to=100,by=10)
for(i in 1:length(ve2))
{
  ve4<-c(ve4,ve2[i],ve3[i])

}
ve4
```

```
##  [1]  3  9 13 19 23 29 33 39 43 49 53 59 63 69 73 79 83 89 93 99
```

```
ve5<-ve[!(ve %in% ve4)]
ve5
```

```
##  [1]   1   2   4   5   6   7   8  10  11  12  14  15  16  17  18  20  21  22  24
## [20]  25  26  27  28  30  31  32  34  35  36  37  38  40  41  42  44  45  46  47
## [39]  48  50  51  52  54  55  56  57  58  60  61  62  64  65  66  67  68  70  71
## [58]  72  74  75  76  77  78  80  81  82  84  85  86  87  88  90  91  92  94  95
## [77]  96  97  98 100
```

Exercise 2

Given the following equation, y=3x^2 + 5x + 2, determine the values of y for an x range of -100 to 100. You might try with and withouth a loop. Plot the function. Hint: use the seq function and try with a step (by) of 1 and 0.1.

```
#Write your code here
x<-c(-100:100)
y1<-c(3*x^2+5*x+2)
y2<-NULL
for(i in 1:length(x)){
  y2<-c(y2,3*x[i]^2+5*x[i]+2)
```

```
}
y1
```

```
##   [1] 29502 28910 28324 27744 27170 26602 26040 25484 24934 24390 23852 23320
##  [13] 22794 22274 21760 21252 20750 20254 19764 19280 18802 18330 17864 17404
##  [25] 16950 16502 16060 15624 15194 14770 14352 13940 13534 13134 12740 12352
##  [37] 11970 11594 11224 10860 10502 10150  9804  9464  9130  8802  8480  8164
##  [49]  7854  7550  7252  6960  6674  6394  6120  5852  5590  5334  5084  4840
##  [61]  4602  4370  4144  3924  3710  3502  3300  3104  2914  2730  2552  2380
##  [73]  2214  2054  1900  1752  1610  1474  1344  1220  1102   990   884   784
##  [85]   690   602   520   444   374   310   252   200   154   114    80    52
##  [97]    30    14     4     0     2    10    24    44    70   102   140   184
## [109]   234   290   352   420   494   574   660   752   850   954  1064  1180
## [121]  1302  1430  1564  1704  1850  2002  2160  2324  2494  2670  2852  3040
## [133]  3234  3434  3640  3852  4070  4294  4524  4760  5002  5250  5504  5764
## [145]  6030  6302  6580  6864  7154  7450  7752  8060  8374  8694  9020  9352
## [157]  9690 10034 10384 10740 11102 11470 11844 12224 12610 13002 13400 13804
## [169] 14214 14630 15052 15480 15914 16354 16800 17252 17710 18174 18644 19120
## [181] 19602 20090 20584 21084 21590 22102 22620 23144 23674 24210 24752 25300
## [193] 25854 26414 26980 27552 28130 28714 29304 29900 30502
```

```
y2
```

```
##   [1] 29502 28910 28324 27744 27170 26602 26040 25484 24934 24390 23852 23320
##  [13] 22794 22274 21760 21252 20750 20254 19764 19280 18802 18330 17864 17404
##  [25] 16950 16502 16060 15624 15194 14770 14352 13940 13534 13134 12740 12352
##  [37] 11970 11594 11224 10860 10502 10150  9804  9464  9130  8802  8480  8164
##  [49]  7854  7550  7252  6960  6674  6394  6120  5852  5590  5334  5084  4840
##  [61]  4602  4370  4144  3924  3710  3502  3300  3104  2914  2730  2552  2380
##  [73]  2214  2054  1900  1752  1610  1474  1344  1220  1102   990   884   784
##  [85]   690   602   520   444   374   310   252   200   154   114    80    52
##  [97]    30    14     4     0     2    10    24    44    70   102   140   184
## [109]   234   290   352   420   494   574   660   752   850   954  1064  1180
## [121]  1302  1430  1564  1704  1850  2002  2160  2324  2494  2670  2852  3040
## [133]  3234  3434  3640  3852  4070  4294  4524  4760  5002  5250  5504  5764
## [145]  6030  6302  6580  6864  7154  7450  7752  8060  8374  8694  9020  9352
## [157]  9690 10034 10384 10740 11102 11470 11844 12224 12610 13002 13400 13804
## [169] 14214 14630 15052 15480 15914 16354 16800 17252 17710 18174 18644 19120
## [181] 19602 20090 20584 21084 21590 22102 22620 23144 23674 24210 24752 25300
## [193] 25854 26414 26980 27552 28130 28714 29304 29900 30502
```

Exercise 3

Given the vectors vA, vB and vC, create a vector vRes that is the sum of all three vectors, i.e., vRes[i] <- vA[i] + vB[i] + vC[i], where max(i) in any case corresponds to the minimum length among the three vectors. Do not use any loop. Remove NAs. Do not hard-code any variable (e.g., you can not code: d <- 4; use instead d <- min(c(5,4,6)).

```
vA <- c(3,5,1,5,6,3,2,1,3,5,2,1,3)
vB <- c(2,4,5,3,7,8,NA,2,4,6,NA,2,4,4,2,9,8,76)
vC <- c(8,5,3,2,4,6,8,9,7,7,0,4,3,5,6,4,3)
```

```
#Write your code here
vB<-c(vB[!is.na(vB)])
vB2<-c(vB[-(length(vA)+1:length(vB))])
vC2<-c(vC[-(length(vA)+1:length(vC))])
vRes<-c(vA+vB2+vC2)
vRes
```

```
##  [1] 13 14  9 10 17 17 12 14 16 14  6  9  8
```

Repeat the exercice but now max(i) corresponds to the maximum length among the three vectors. Same conditions as above apply. A non-existing value in any vector should be computed as 0.

```
#Write your code here
VA2<-vA
vB2<-vB
for (i in (length(VA2)+1):length(vC)){
  VA2 <- c(VA2,0)
}
for (i in (length(vB2)+1):length(vC)){
  vB2 <- c(vB2,0)
}

vRes2<-c(VA2+vB2+vC)
vRes2
```

```
##  [1] 13 14  9 10 17 17 12 14 16 14  6  9  8 14 14 80  3
```

Take a look at the functions rbind and cbind. Construct a matrix appending the three vectors as its columns or rows. Use resulting appended column matrix and loop across its rows. Find the column number storing the highest value and store its value and index (column number) in a two-column matrix.

```
#Write your code here

MA2<-NULL
MA<-cbind(VA2,vB2,vC)
for (i in 1:nrow(MA)){
  max<-0
  for(j in 1:ncol(MA)){
    if(max<MA[i,j]) {
      max<-MA[i,j]
      index<-j

    }

  }
  MA2<-rbind(MA2,cbind(max,index))
}

MA
```

```
##       VA2 vB2 vC
```

```
##  [1,]   3   2  8
##  [2,]   5   4  5
##  [3,]   1   5  3
##  [4,]   5   3  2
##  [5,]   6   7  4
##  [6,]   3   8  6
##  [7,]   2   2  8
##  [8,]   1   4  9
##  [9,]   3   6  7
## [10,]   5   2  7
## [11,]   2   4  0
## [12,]   1   4  4
## [13,]   3   2  3
## [14,]   0   9  5
## [15,]   0   8  6
## [16,]   0  76  4
## [17,]   0   0  3
```

```
MA2
```

```
##     max index
## vC    8     3
## VA2   5     1
## vB2   5     2
## VA2   5     1
## vB2   7     2
## vB2   8     2
## vC    8     3
## vC    9     3
## vC    7     3
## vC    7     3
## vB2   4     2
## vB2   4     2
## VA2   3     1
## vB2   9     2
## vB2   8     2
## vB2  76     2
## vC    3     3
```